

EXERCISE

- rewrite inner for as a list comprehension

```
HwyMpgByClass = []
```

```
for t in vehicleclass:
```

```
    list_summpg = [float(d['hwy']) for d in mpg if d['class'] == t]
```

```
    HwyMpgByClass.append((t, sum(list_summpg) / len(list_summpg)))
```

- both for as a list comprehension

```
def avg_mpg(t):
```

```
    list_summpg = [float(d['hwy']) for d in mpg if d['class'] == t]
```

```
    return sum(list_summpg) / len(list_summpg)
```

```
HwyMpgByClass = [(t, avg_mpg(t)) for t in vehicleclass]
```

INTRODUCTION TO NUMPY

MOTIVATION

- What if we want to do element-wise addition over two lists?

```
>>> a=[1, 2, 3]
```

```
>>> b=[4,5,6]
```

```
>>> a+b
```

```
[1, 2, 3, 4, 5, 6]
```

→ this is concatenation

- Use **zip** function

```
>>> result = []
```

```
>>> for first, second in zip(a, b):
```

```
...     result.append(first+second)
```

```
[5, 7, 9]
```

→ annoying code

- Grows in complexity with multidimensional arrays
- Very slow

IMPORT

import numpy **as** np

- when you see **np** you should think NumPy
- all after np comes from the NumPy package
- for example, create a NumPy array

```
>>> a=np.array([1,2,3])
```

ARRAYS

- for this introduction we will see 1 and 2 dimensional arrays
- array constructor

```
>>> a = np.array([1,2,3])
```

```
>>> a.dtype
```

```
dtype('int32')
```

- the first part is integer, and the second part is how many bits represents this integer
- reducing precision

```
>>> a = np.array([1,2,3], dtype='int8')
```

```
>>> a.dtype
```

```
dtype('int8')
```

- all the data in the array has to be of the same type

```
>>> a[0] = 10.5
```

```
array([10, 2, 3], dtype=int8)
```

float in array of integers
the number is truncated

OPERATORS

- element-wise operators between arrays

```
>>> a = np.array([1.2, 2.3, 3.4, 4.5])
```

```
>>> b = np.array([5.6, 6.7, 7.8, 9.1])
```

- $a + b$  `array([ 6.8, 9. , 11.2, 13.6])`
- $a - b$  `array([-4.4, -4.4, -4.4, -4.6])`
- $a * b$  `array([ 6.72, 15.41, 26.52, 40.95])`
- a / b  `array([0.21428571, 0.34328358, 0.43589744, 0.49450549])`
- $a ** b$  `array([2.77597134e+00, 2.65201821e+02, 1.39809136e+04, 8.79496014e+05])`

OPERATORS

- operators between an array and a constant

```
>>> a=np.array([1.2, 2.3, 3.4, 4.5])
```

```
>>> b= 4
```

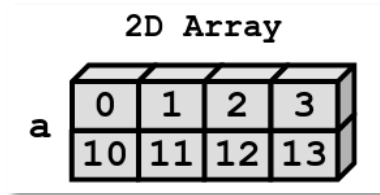
- $a + b$  `array([5.2, 6.3, 7.4, 8.5])`
- $a - b$  `array([-2.8, -1.7, -0.6, 0.5])`
- $a * b$  `array([ 4.8, 9.2, 13.6, 18.])`
- a / b  `array([0.3 , 0.575, 0.85 , 1.125])`
- $a ** b$  `array([ 2.0736, 27.9841, 133.6336, 410.0625])`

ARRAYS

```
>> a = np.array([[ 0, 1, 2, 3],  
                  [10,11,12,13]])
```

```
>> a
```

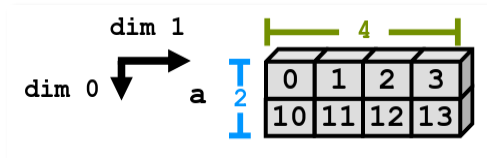
```
array([[ 0, 1, 2, 3],  
       [10,11,12,13]])
```



- `shape = (rows, columns)`

```
>> a.shape
```

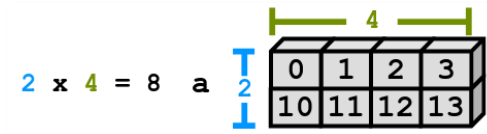
```
(2, 4)
```



- `element count`

```
>> a.size
```

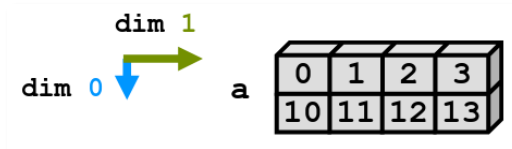
```
8
```



- `number of dimensions`

```
>> a.ndim
```

```
2
```



ARRAYS

- get/set elements

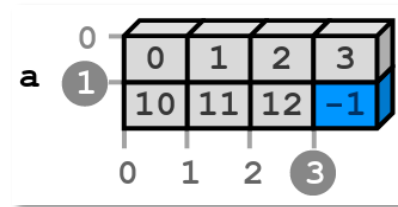
```
>> a[1, 3]
```

```
13
```

```
>> a[1, 3] = -1
```

```
>> a
```

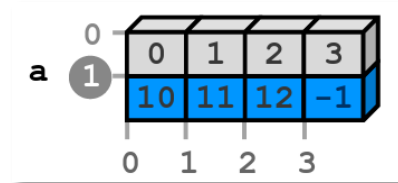
```
array([[ 0,  1,  2,  3],  
       [10, 11, 12, -1]])
```



- Address second row using single index

```
>> a[1]
```

```
array([10, 11, 12, -1])
```



SLICING

- **var[lower : upper : step]**
- extracts a portion of a sequence by specifying a **lower** and **upper** bound
- the **lower bound** element **is included**, but the **upper-bound** element **is not included**: [lower, upper)
- the **step** value specifies the stride between elements

```
>>> a = np.array([10, 11, 12, 13, 14])
```

```
>>> a[1 : 3]
```

```
array([ 11, 12])
```

- negative indices work as well

```
>>> a[1 : -2]
```

```
array([ 11, 12])
```

- omitted boundaries are assumed to be from the beginning

```
>>> a[: 3]
```

```
array([10, 11, 12])
```

```
>>> a[-2 :]
```

```
array([13, 14])
```

- every two elements

```
>>> a[:: 2]
```

```
array([10, 12, 14])
```

SLICING

- get all orange

```
>>> a[0, 3:5]
```

```
array([3, 4])
```

- get all blue

```
>>> a[4:, 4:]
```

```
array([[44, 45],  
       [54, 55]])
```

- get all red

```
>>> a[:, 2]
```

```
array([2, 12, 22, 32, 42, 52])
```

- stride is also possible

```
>>> a[2::2, ::2]
```

```
array([[20, 22, 24],
```

```
       [40, 42, 44]])
```

	0	1	2	3	4	5
0	0	1	2	3	4	5
1	10	11	12	13	14	15
2	20	21	22	23	24	25
3	30	31	32	33	34	35
4	40	41	42	43	44	45
5	50	51	52	53	54	55

SLICES ARE REFERENCES

- slices are references to locations in memory
- these memory locations can be used in assignment operations

```
>>> a = np.array([10, 11, 12, 13, 14])
```

```
>>> a[-2 : ]
```

```
array([ 13, 14])
```

- we can insert an iterable of length two

```
>>> a[-2 : ] = [ -1, -2 ]
```

```
array([10, 11, 12, -1, -2])
```

- Or scalar value

```
>>> a[-2 : ] = 99
```

```
array([10, 11, 12, 99, 99])
```

SLICING EXERCISE

- create the array below with the command

```
>>> a = np.arange(25).reshape(5,5)
```

- get all orange

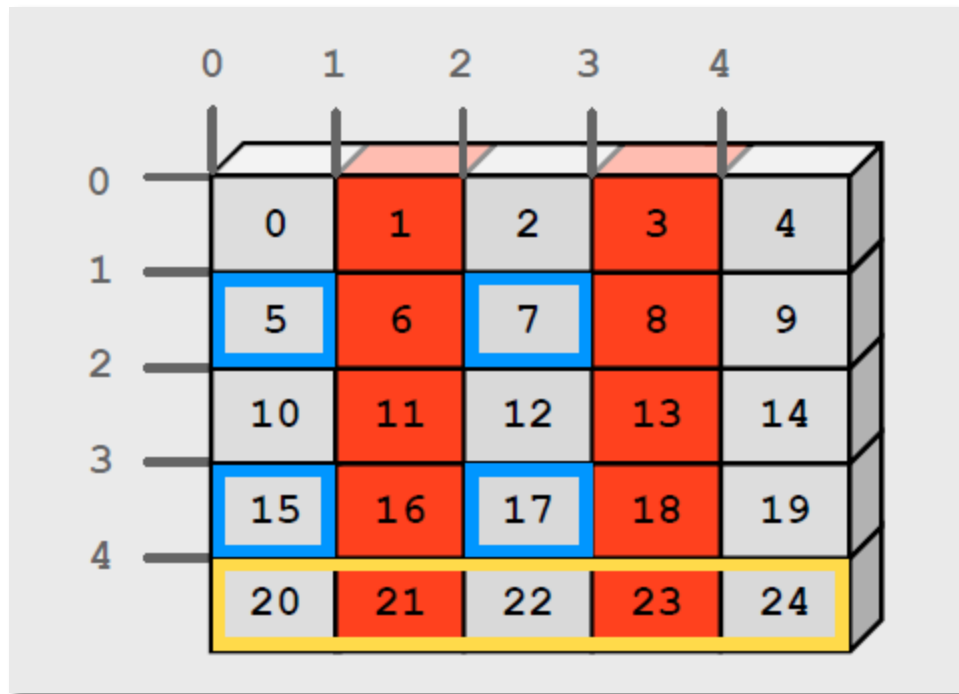
```
>>> ...
```

- get all blue

```
>>> ...
```

- get all red

```
>>> ...
```



ARRAY CREATION FUNCTIONS

- array of values in the range [start, stop): **arange([start], stop, [, step], dtype=None)**

```
>>> np.arange(4)
```

```
array([0, 1, 2, 3, 4])
```

```
>>> a = np.arange(0,5,2)
```

```
array([0, 2, 4])
```

- array of ones: **np.ones(shape, dtype='float64')**

```
>>> np.ones((2,3))
```

```
array([[1., 1., 1.],  
       [1., 1., 1.]])
```

- array of zeros: **np.zeros(shape, dtype='float64')**

```
>>> np.zeros((2,3))
```

```
array([[0., 0., 0.],  
       [0., 0., 0.]])
```

ARRAY CREATION FUNCTIONS

- identity array

```
>>> np.identity( 4 )
```

```
array([[1., 0., 0., 0.],  
       [0., 1., 0., 0.],  
       [0., 0., 1., 0.],  
       [0., 0., 0., 1.]])
```

- array filled with a number

```
>>> np.full( 2, 7.0 )
```

```
array([7., 7.])
```

- array of N evenly spaced elements between (and including) start and stop values

```
>>> np.linspace(0, 1, 5)
```

```
array([0. , 0.25, 0.5 , 0.75, 1.  ])
```

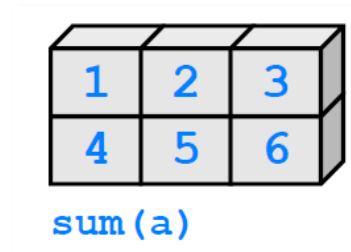
ARRAY CALCULATION

```
>> a = np.array([[ 1, 2, 3 ],  
                  [ 4,5,6]])
```

- sum all the values in array

```
>>> a.sum()
```

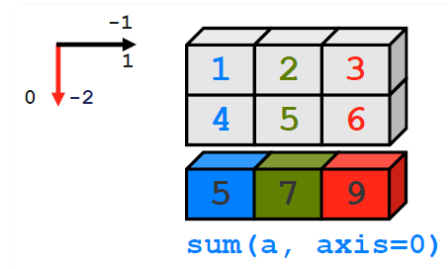
```
21
```



- supply the keyword axis to sum along the 0th axis

```
>>> a.sum(axis=0)
```

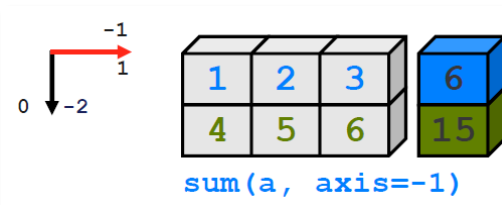
```
array([5, 7, 9])
```



- supply the keyword axis to sum along the last axis

```
>>> a.sum(axis=-1)
```

```
array([ 6, 15])
```



OPERATIONS ON ARRAYS

- mathematical functions: sum, prod, min, max, argmin, argmax, ...

```
>>> a = np.arange(10, 20)
```

```
array([10, 11, 12, 13, 14, 15, 16, 17, 18, 19])
```

```
>> a.argmin()
```

```
0
```

```
>> a.argmax()
```

```
9
```

- statistics: mean, std, ...

OPERATIONS ON ARRAYS EXERCISE

- create the array below with the command

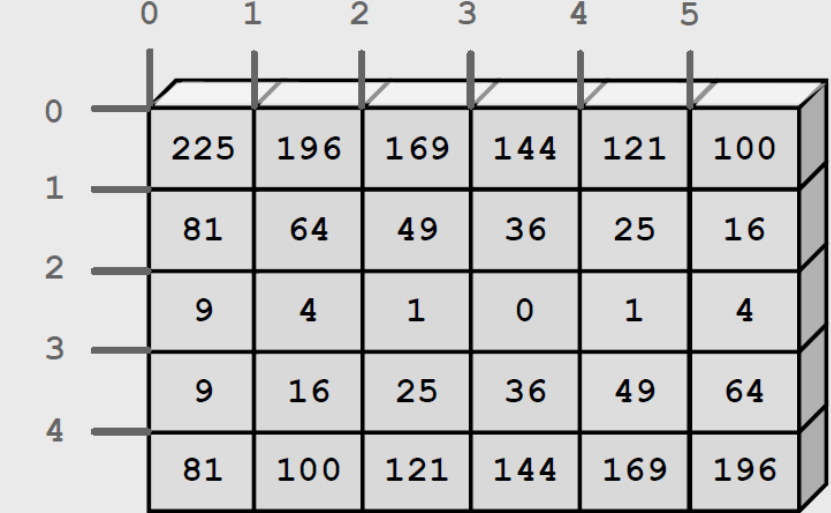
```
a = np.arange(-15, 15).reshape(5, 6) ** 2
```

- compute the maximum of each row (one max per row).

```
>>> ...
```

- the mean of each column (one mean per column)

```
>> ...
```



	0	1	2	3	4	5
0	225	196	169	144	121	100
1	81	64	49	36	25	16
2	9	4	1	0	1	4
3	9	16	25	36	49	64
4	81	100	121	144	169	196

MATPLOTLIB

- reference: <https://matplotlib.org/stable/>

PLOT

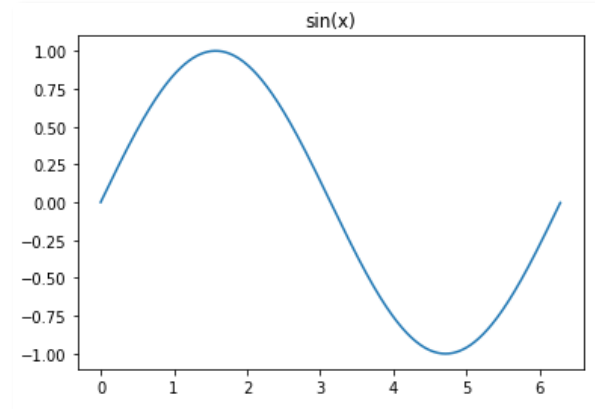
- plot y versus x using a line

```
import numpy as np
import matplotlib.pyplot as plt
```

```
X = np.arange(0, 2*np.pi, 0.01)
```

```
Y = np.sin(X)
```

```
plt.plot(X, Y)
plt.title('sin(x)')
```



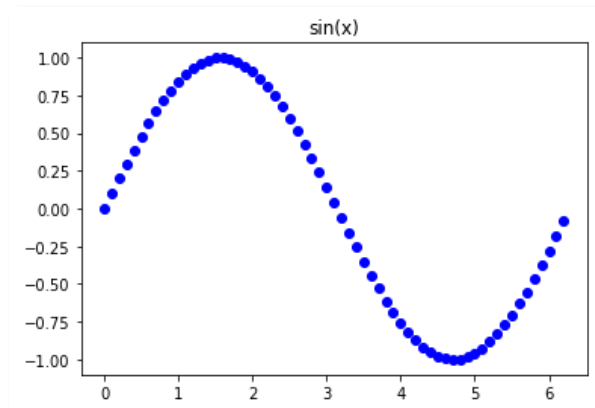
- plot y versus x using markers

```
import numpy as np
import matplotlib.pyplot as plt
```

```
X = np.arange(0, 2*np.pi, 0.1)
```

```
Y = np.sin(X)
```

```
plt.plot(X, Y, 'bo') # markers
plt.title('sin(x)')
```



PLOT

```
plt.plot(X,np.sin(X),label='sin(x)')
```

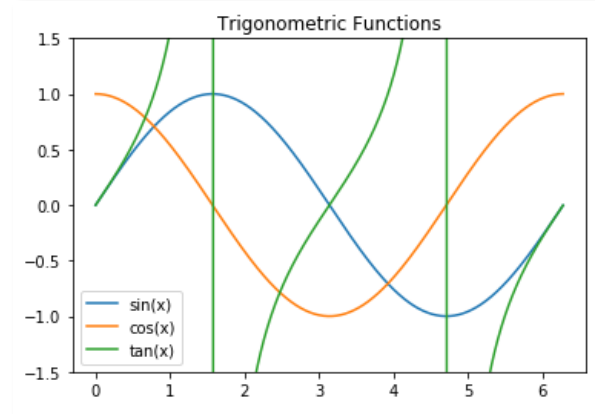
```
plt.plot(X,np.cos(X),label='cos(x)')
```

```
plt.plot(X,np.tan(X),label='tan(x)')
```

```
plt.title('Trigonometric Functions')
```

```
plt.ylim(-1.5,1.5) # limit the range of y values
```

```
plt.legend() # show the legend
```



HISTOGRAMS

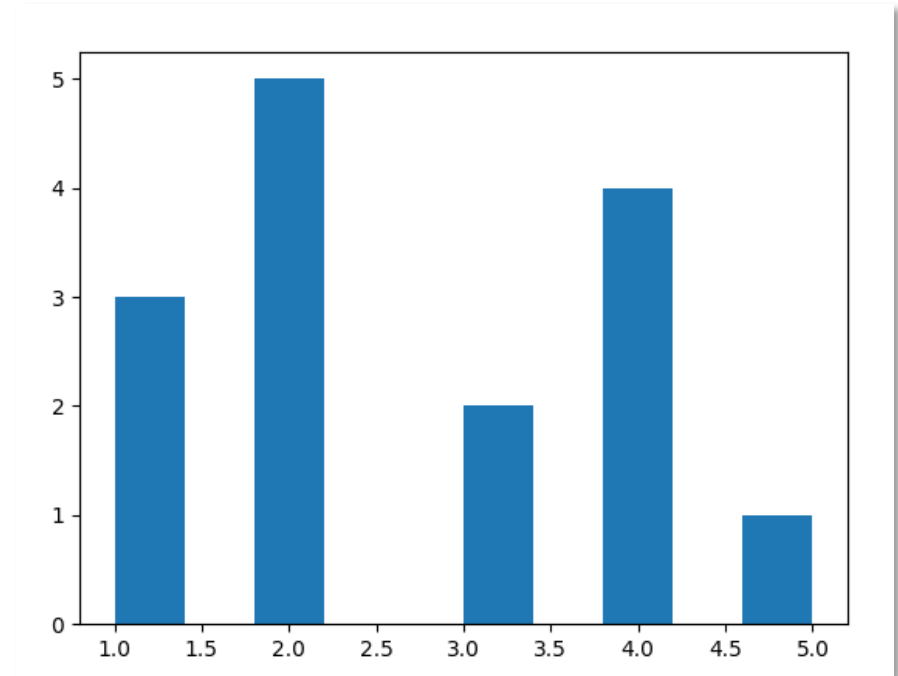
- suppose that we have the following data

```
data = [1, 1, 1, 2, 2, 2, 2, 2, 3, 3, 4, 4, 4, 4, 5]
```

- plot how many 1's, 2's, 3's, ... we have in the data

```
import matplotlib.pyplot as plt
```

```
plt.hist(data)
```



- all the bars are touching, with no space between them
- the bars represent the quantity of values that fall between each 'bin' or range of values

HISTOGRAMS

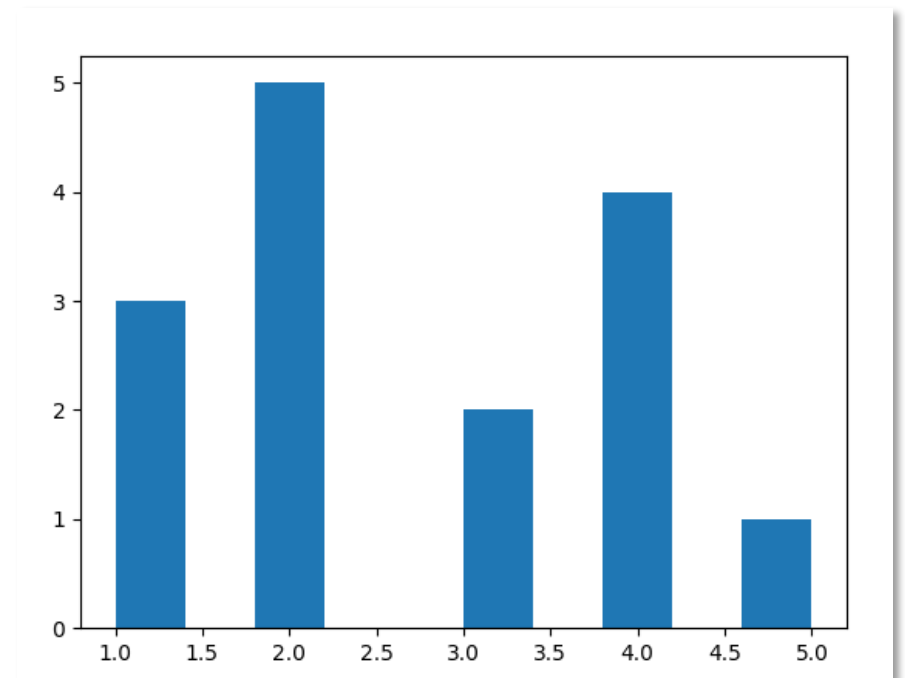
- get the bins of the histogram

```
_, bins, _ = plt.hist(data)
```

```
print(bins)
```

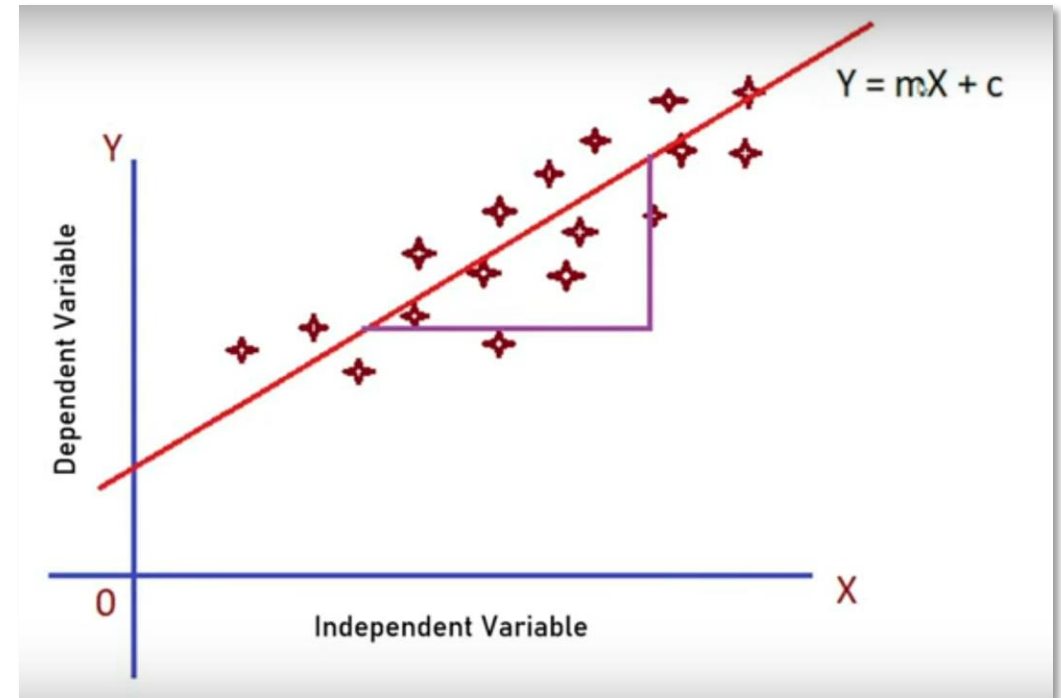
```
[1. 1.4 1.8 2.2 2.6 3. 3.4 3.8 4.2 4.6 5.]
```

- the first bin is [1., 1.4) (including 1., but excluding 1.4)
- the second bin is [1.4, 1.8) (including 1.4, but excluding 1.8)
- the last bin is [4.6, 5.] (including 4.6, and 5.)



LINEAR REGRESSION

- if X and Y have some correlation
- from X we can estimate Y
- search m and c that best fit the data
- m is the slope of the line
- c is the intersection



LINEAR REGRESSION

```
import numpy as np

from sklearn.linear_model import LinearRegression

x = np.linspace(0, 10, 21)
y = 10 + x

noise = np.random.normal(scale=0.3, size=y.shape)
y += noise

x = x.reshape(len(x), 1)
y = y.reshape(len(y), 1)

reg = LinearRegression().fit(x, y)
Y = reg.coef_ * x + reg.intercept_

plt.plot(x, y, "bo", label="data")
plt.plot(x, Y, "red", label="linear regression, Y = Xm + c")
plt.legend()
```

